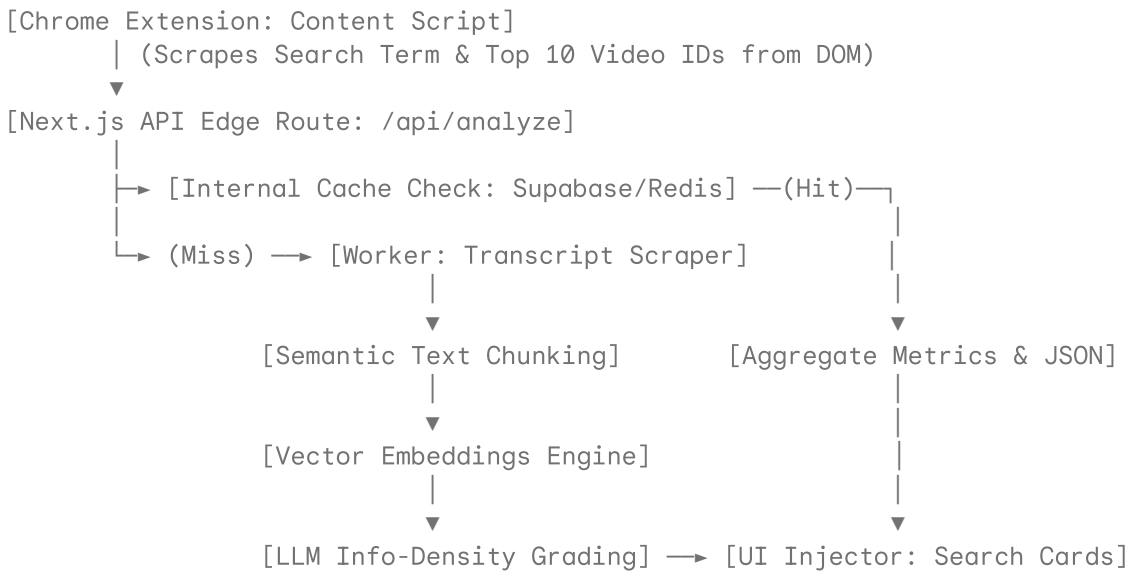
🚀 EchoFilter: AI-Powered Transcript Mining & Info-Density Engine

EchoFilter is a developer-focused Chrome Extension and Next.js backend infrastructure that bypasses traditional YouTube SEO algorithms. Instead of relying on misleading vanity metrics (likes, views, clickbait thumbnails), EchoFilter downloads video transcripts asynchronously, breaks them down into semantic temporal windows, vectors them against user search intent, and runs a custom **Information Density Matrix** to rank videos by actual technical utility.

🛠️ System Architecture & Data Flow



📂 Core Data Structures & Type Definitions

These TypeScript interfaces define the absolute contract between the Extension Frontend, the Processing Backend, and the Database layer.

1. Payload From Extension to Backend (`/api/analyze`)

```
interface AnalysisPayload {
  userId: string;
  searchQuery: string;
  videoIds: string[]; // Batched top 5-10 video IDs visible in DOM
}
```

2. Internal Transcript Chunk Schema

```
interface TranscriptChunk {
  id: string;          // Format: ${videoId}_ch_${index}
  videoId: string;
  text: string;        // Normalized and cleaned transcript text
  startTime: number;   // In seconds
```

```
  duration: number;    // Window duration in seconds
  embedding?: number[]; // 384-dimensional vector array (if using local models)
}
```

3. Final JSON Analysis Response

```
interface VideoDensityReport {
  videoId: string;
  metrics: {
    infoDensityScore: number;  // Scale 0.0 - 1.0 (Percentage representation)
    fillerRatio: number;       // Ratio of generic fluff/sponsors to hard facts
    confidenceScore: number;   // ML alignment score
  };
  keyTimestamps: {
    timeInSeconds: number;
    relevanceReason: string;   // Contextual snippet summarizing why this times
  }[];
  verdict: "HIGH_DENSITY" | "SURFACE_LEVEL" | "CLICKBAIT_WASTE";
}

interface AnalyticsBatchResponse {
  searchQuery: string;
  results: Record<string, VideoDensityReport>; // Keyed by Video ID for O(1) fr
}
```

🧠 Advanced Extraction Pipelines & AI Routing

To maintain a processing speed under 2 seconds per batch, execution runs as a hybrid pipeline: a fast **Embedding Model** handles structural filtering, followed by a deterministic, structured-output **LLM Call** (via Groq/Llama-3 or OpenAI GPT-4o-mini) to calculate density metrics.

Pipeline Stage 1: Vector Alignment

For each chunk, calculate the Cosine Similarity against the `searchQuery`:

$$\text{Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

Only chunks matching a threshold of ≥ 0.55 are passed to Stage 2 to optimize token consumption and prevent context window bloat.

Pipeline Stage 2: System Prompt (Structured JSON Enforcement)

This exact system prompt forces the LLM to output pure JSON without conversational padding, calculating code density and identifying fluff.

```
You are an elite, highly critical Technical Code Auditor. Your job is to evalua

[User Query]: {{searchQuery}}
[Transcript Segment]: {{transcriptSegment}}

Analyze the transcript segment objectively. You must score the text based purel
```

```
You must return a raw, syntactically flawless JSON object matching the schema b

JSON Schema:
{
  "technicalScore": <float between 0.00 and 1.00 indicating raw factual content
  "fillerDetected": <float between 0.00 and 1.00 indicating fluff/sponsor conte
  "justification": "<string; 15-word maximum summarizing the specific technical
}
```

Backend API Endpoints

1. Execute Video Processing Engine

- **Endpoint:** `POST /api/analyze`
- **Headers:** `Content-Type: application/json`

Sample Request Body:

```
{
  "userId": "usr_94f8ba11cde2",
  "searchQuery": "NextJS middleware WebGL context loss recovery",
  "videoIds": ["dQw4w9WgXcQ", "z9750vX_f7I"]
}
```

Sample Response Body (200 OK):

```
{
  "searchQuery": "NextJS middleware WebGL context loss recovery",
  "results": {
    "z9750vX_f7I": {
      "videoId": "z9750vX_f7I",
      "metrics": {
        "infoDensityScore": 0.89,
        "fillerRatio": 0.08,
        "confidenceScore": 0.94
      },
      "keyTimestamps": [
        {
          "timeInSeconds": 342,
          "relevanceReason": "Code walkthrough implementing canvas event listen
        }
      ],
      "verdict": "HIGH_DENSITY"
    },
    "dQw4w9WgXcQ": {
      "videoId": "dQw4w9WgXcQ",
      "metrics": {
        "infoDensityScore": 0.12,
        "fillerRatio": 0.85,
        "confidenceScore": 0.99
      },
      "keyTimestamps": [],
      "verdict": "CLICKBAIT_WASTE"
    }
```

```
  }
}
```

2. Direct Crowd-Sourced Badge Tagging (Extension-Driven)

- **Endpoint:** `POST /api/tag`
- **Headers:** `Content-Type: application/json`

Allows the extension community to override or confirm the AI's data model, ensuring a decentralized validation network.

Sample Request Body:

```
{
  "videoId": "z9750vX_f7I",
  "tag": "OUTDATED_DEPENDENCIES",
  "userId": "usr_94f8ba11cde2"
}
```

Sample Response Body (201 Created):

```
{
  "status": "success",
  "message": "Community tag recorded. Security weights updated."
}
```

🧮 Custom Information-Density Ranking Algorithm

When rendering search components on the client side, the final score (S_{final}) determining whether a video card gets brought to the top of the YouTube feed is computed using this deterministic equation:

$$S_{\text{final}} = (w_1 \cdot D_{\text{info}}) - (w_2 \cdot R_{\text{filler}}) + (w_3 \cdot \log_{10}(C_{\text{community}}))$$

Where:

- D_{info} = Base Info-Density Score $(0.0 - 1.0)$
- R_{filler} = Filler/Fluff Ratio $(0.0 - 1.0)$
- $C_{\text{community}}$ = Verified community confirmations upvoting the AI's accuracy rating.
- Weights are strictly set to: $w_1 = 0.65, w_2 = 0.25, w_3 = 0.10$.

⚡ Setup & Production Deployment

1. Production Environment Variables (`.env.local`)

```
NEXT_PUBLIC_SUPABASE_URL=[https://your-project.supabase.co](https://your-projec
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
GROQ_API_KEY=gsk_vA901b...
UPSTASH_REDIS_REST_URL=[https://your-redis-instance.upstash.io](https://your-re
UPSTASH_REDIS_REST_TOKEN=AbCdEfGhIj...
```

2. Run Locally

```
# Clone and install dependencies
git clone [https://github.com/yourusername/echofilter.git](https://github.com/y
cd echofilter
npm install

# Run the development server for Next.js API infrastructure
npm run dev
```

3. Loading the Extension

1. Open Chrome and navigate to `chrome://extensions/` .
2. Enable **Developer mode** (top-right toggle switch).
3. Click **Load unpacked** in the upper-left corner.
4. Select the `/extension` directory containing your compiled `manifest.json` and production scripts.